# Ch3. 쿠버네티스로 컨테이너 실행하기

- kubectl
  - 쿠버네티스를 관리하는 기본 커맨드라인 인터페이스

---

## kubectl

- 쿠버네티스 클러스터를 관리하는 동작 대부분은 kubectl이라는 CLI로 실행할 수 있다.
- kubectl이 지원하는 명령
  - 쿠버네티스 자원들의 생성, 업데이트, 삭제(create, update, delete)
  - 디버그, 모니터링, 트러블 슈팅(log, exec, cp, top, attach, ...)
  - 클러스터 관리(cordon, top, drain, taint, ...)

### 1. 설치

### 2. 기본 사용법

```
kubectl 명령 타입 이름 플래그
```

- 명령: 자원에 실행하려는 동작. create, get, delete 등
- 타입: 자원타입. pod, service, ingress 등
- 이름: 자원이름
- 플래그: 부가적으로 설정할 옵션

[실습]

echoserver라는 이름의 파드 생성

```
$ kubectl run echoserver --image="cilium/echoserver:1.10" --p
```

kubectl get: 쿠버네티스에 있는 자원 상태를 확인할 때 가장 많이 사용하는 명령

- get 뒤에는 확인하려는 자원 이름과 옵션을 설정해 자원 상태를 확인

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
echoserver    1/1     Running   0          7m21s
```

실행 결과의 항목 각각

- NAME: 파드의 이름을 표시
- READY: 숫자/숫자 형태로 파드의 준비 상태를 표시
  - 0/1이면 현재 파드는 생성되었으나 사용할 준비가 되지 않았다는 뜻
  - 1/1이면 파드가 생성되었고, 사용할 준비가 끝났다는 뜻
- STATUS: 파드의 현재 상태
  - Running: 파드가 실행되었다는 뜻
  - Terminating: 컨테이너 삭제 중
  - ContainerCreating: 컨테이서 삭제 중
- RESTARTS: 해당 파드가 몇번 재시작했는지를 표시
- AGE: 파드를 생성한 후 얼마나 시간이 지났는지 표시

```
kubectl expose pods echoserver --name=echoserver-svc --type=N
```

- 파드에 접근할 수 있도록 서비스를 생성한 후 확인

```
$ kubectl get services
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   P
echoserver-svc   NodePort    10.106.111.132   <none>        8
kubernetes       ClusterIP   10.96.0.1        <none>        4
```

- NAME: 서비스 이름 표시
- TYPE: 서비스 타입
- CLUSTER-IP: 현재 클러스터 안에서 사용되는 IP
- EXTERNAL-IP: 클러스터 외부에서 접속할 때 사용하는 IP. 별도로 설정하지 않았으므로 <none>
- PORT(S): 해당 서비스에 접속하는 포트를 표시

- AGE: 자원을 생성한 후 얼마나 시간이 지났는지 표시

파드의 로그를 수집할 때

```
kubectl logs 파드명
```

```
$ kubectl logs echoserver
Generating self-signed cert
Generating a 2048 bit RSA private key
.....+++
.........................................+++
writing new private key to '/certs/privateKey.key'
-----
Starting nginx
```

- 시간, HTTP 버전, 웹 브라우저와 운영체제 버전 등의 정보를 확인

```
kubectl delete pod 파드명
kubectl delete service 서비스명
```

- 파드, 서비스 삭제

```
kubectl delete pod echoserver
kubectl delete service echoserver
```

# 3. POSIX/GNU 스타일의 명령 작성 규칙

kubectl은 기본적으로 POSIX/GNU 스타일의 명령 작성 규칙을 따른다.

- -과 함께 사용하는 옵션은 단일 알파벳/숫자 문자 인자는 짧은 옵션이다.
- 일부 옵션은 인자를 필요로 한다.
- --과 함께 사용하는 옵션은 알파벳 두 글자 이상으로 구성한 긴 옵션이다.
- -- 이후에 작성하는 인자가 있다면 쿠버네티스 관련 옵션들을 모두 종료한다.

# 4. 플래그

- 전역 플래그: 모든 명령에서 사용

    - kubectl options에서 확인
- 개별 플래그: 개별 명령에서만 사용
    - 개별 플래그는 명령 각각의 도움말에서 확인
- 기억해둘만한 전역 플래그
    - **-h(--help)**: kubectl 명령어 --help 형태로 사용. 개별 명령의 도움말을 출력
    - **-v 로그레벨**: 명령을 실행하는 과정의 로그를 출력하거나 로그 레벨을 출력. 디버깅 할 때 유용

## 5. kubeconfig 환경 변수

- kubectl은 기본적을 $HOME/.kube/config 파일에서 클러스터, 인증, 컨텍스트 정보를 읽어 들인다.
- 클러스터 구성 정보를 **kubeconfig**라 한다.

**kubectl api-resources**

```
kubectl api-resources | more
NAME                              SHORTNAMES   APIVERSION
KIND
bindings                                       v1
 Binding
componentstatuses                 cs           v1
 ComponentStatus
configmaps                        cm           v1
 ConfigMap
...
```

- 클러스터에서 사용할 수 있는 자원들을 확인
    - SHORTNAME: 자원의 단축 이름
    - APIGROUP: 함께 노출되는 자원 그룹
    - NAMESPACED: 특정 네임스페이스에 속하는 자원인지
    - KIND: 객체 스키마

## 6. 자동 안성

kubectl은 배시 쉘에서의 자동 완성을 공식적으로 지원한다.

```
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc # kubectl 명령어를 k 로 사용
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc
```

- kubectl completion --help 명령을 실행하면 자동 완성 기능을 설정하는 방법을 소개

---

# 디플로이먼트를 이용해 컨테이너 실행하기

쿠버네티스를 이용해서 디플로이먼트를 실행하는 방법 2가지

- **kubectl create 명령**으로 직접 디플로이먼트를 실행하는 것
- 디플로이먼트 내의 파드를 어떻게 실행할지 세부 사항을 담은 **YAML 형식의 템플릿**으로 디플로이먼트를 실행하는 것